

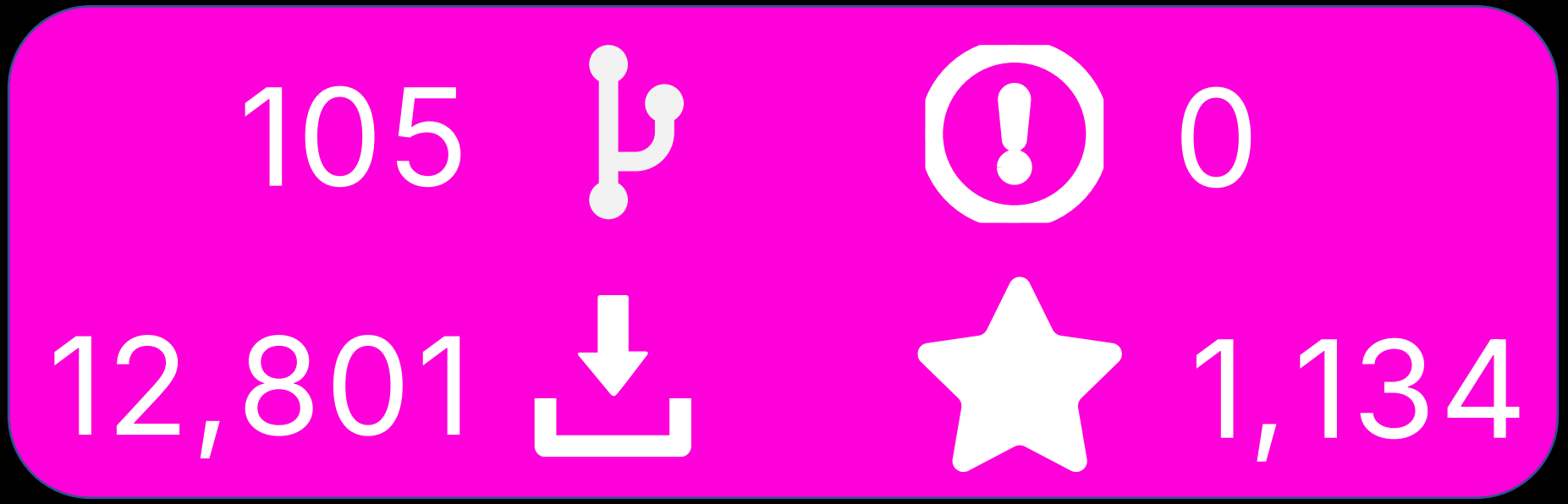
OFF-BY-ONE
2025

SPEED UP YOUR REVERSE ENGINEERING WITH THE HRTNG PLUGIN

GEORGY KUCHERIN
KASPERSKY GREAT

THE HRTNG IDA PRO PLUGIN

Goal:
Make reverse engineering simpler and faster



github.com/KasperskyLab/hrtng
hrtng – Hex-Rays Tools Next Generation



OFF-BY-ONE
2025

PLUGIN HISTORY

2016

Development started

2024

Code published (GPLv3 license)

2025

Hex-Rays contest winner



OFF-BY-ONE
2025

OUR SAMPLE FOR TODAY

FinSpy
commercial
spyware

First use cases
more than 15 years ago

Lots of obfuscation
found in code

Available for
many platforms



OFF-BY-ONE
2025

OUR SAMPLE FOR TODAY

Kaspersky Lab's advanced exploit prevention system has identified a new Adobe Flash zero day exploit, used in an attack on 10 October by a threat actor known as BlackOasis. The exploit is delivered through a Microsoft Word document and deploys the FinSpy commercial malware. Kaspersky Lab has reported the vulnerability to Adobe, which has issued an advisory.

CVE-2017-0199 Used as Zero Day to Distribute FINSPY Espionage Malware and LATENTBOT Cyber Crime Malware



OFF-BY-ONE
2025

THE FIRST INSTRUCTIONS

```
4D          dec     ebp
5A          pop     edx
B0 01       mov     al, 1
52          push    edx
```

Junk instructions



OFF-BY-ONE
2025

THE FIRST INSTRUCTIONS

```
B9 B2 C7 02 00   mov     ecx, 2C7B2h
53               push    ebx
BB 5E BB E4 F6   mov     ebx, 0F6E4BB5Eh
57               push    edi
D9 EE            fldz
9B D9 74 24 E4   fstenv  byte ptr [esp-1Ch]
8B 7C 24 F0      mov     edi, [esp-10h]
8D 57 1D         lea     edx, [edi+1Dh]
```



OFF-BY-ONE
2025

THE FIRST INSTRUCTIONS

32-Bit Protected Mode Format

31 … 16	15 … 0	
	Control Word	0
	Status Word	4
	Tag Word	8
FPU Instruction Pointer Offset (FIP)		12
0 0 0 0 0 | Bits 10:0 of opcode	FPU Instruction Pointer Selector	16
FPU Data Pointer Offset (FDP)		20
	FPU Data Pointer Selector (FDS)	24

For instructions that also store x87 FPU data registers, the eight 80-bit registers (R0-R7) follow the above structure in sequence.

Protected Mode x87 FPU State Image in Memory, 32-Bit Format



THE FIRST INSTRUCTIONS

```
struct sFPUstate {
  int cw; // esp - 1Ch
  int sw; // esp - 18h
  int tw; // esp - 14h
  int fip; // esp - 10h - address of fldz
  int fis;
  int fdp;
  int fds;
};
```

```
fldz
fstenv  byte ptr [esp-1Ch]
mov     edi, [esp-10h]
```



THE FIRST INSTRUCTIONS

```
8B 7C 24 F0      mov     edi, [esp-10h]
8D 57 1D         lea     edx, [edi+1Dh]
```

```
loc_40001F:
30 5C 0A FF      xor     [edx+ecx-1], bl
D1 C3            rol     ebx, 1
E2 F8            loop    loc_40001F
```

DECRYPTION LOOP



OFF-BY-ONE
2025

DECRYPTING DATA

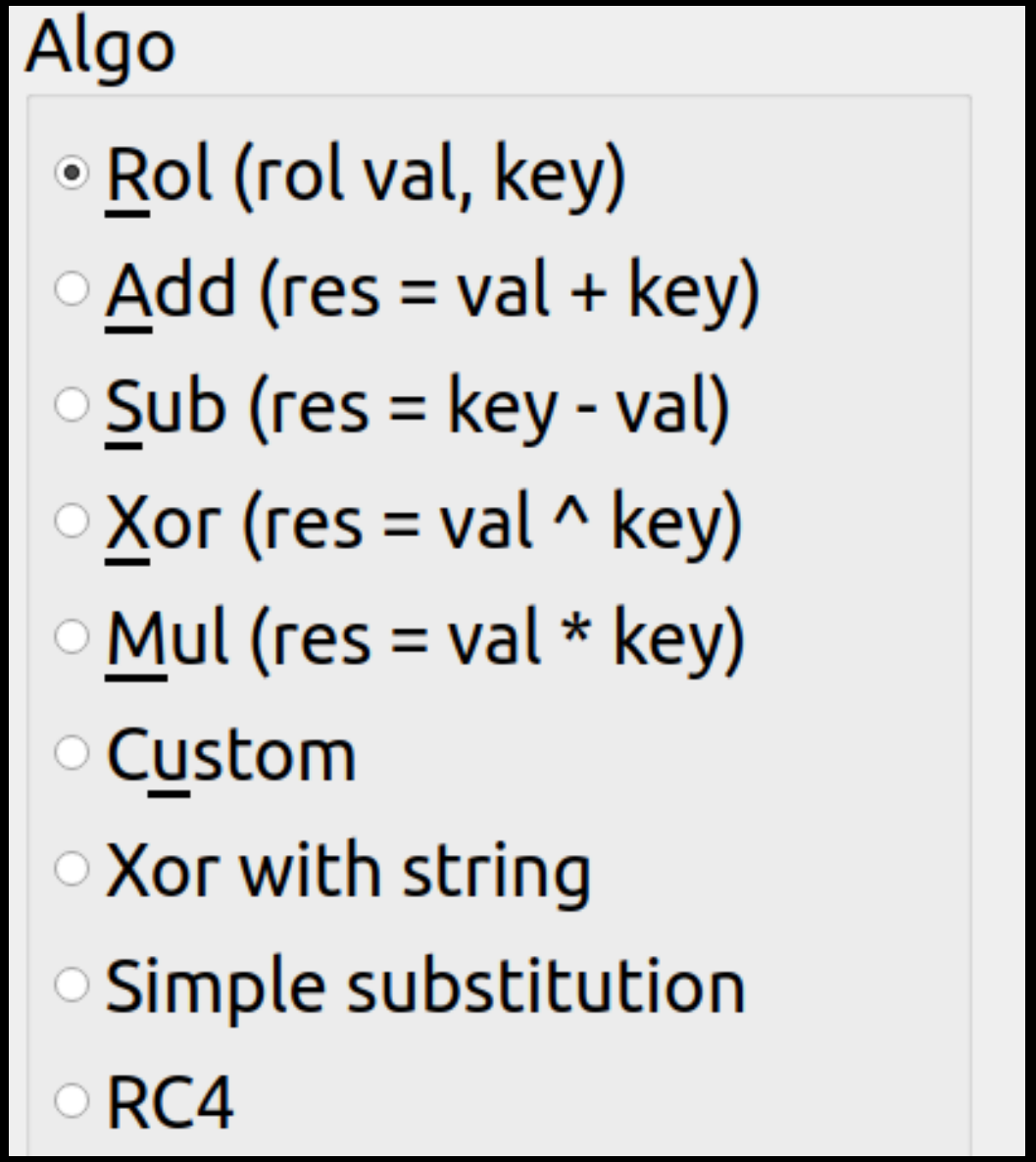
```
loc_40001F:
30 5C 0A FF     xor     [edx+ecx-1], bl
D1 C3           rol      ebx, 1
E2 F8           loop    loc_40001F
```

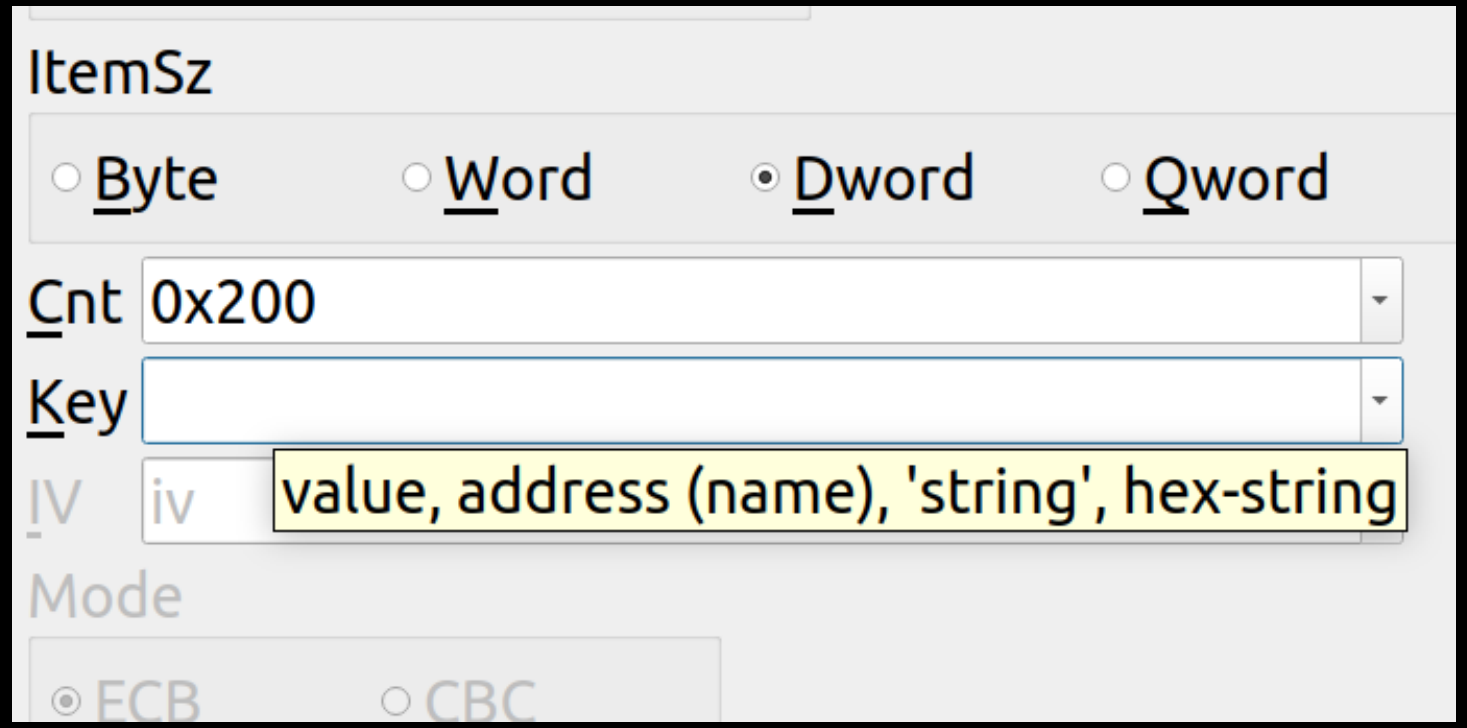
```
seg000:0000002E                 db  20h
seg000:0000002F                 db  83h
seg000:00000030                 db 0DEh
seg000:00000031                 db 0B9h
seg000:00000032                 db 0DCh
seg000:00000033 unk_33          db  87h
seg000:00000034                 db  22h ; "
seg000:00000035                 db  74h ; t
seg000:00000036                 db 0EDh
seg000:00000037                 db 0F6h
```



OFF-BY-ONE
2025

DECRYPTING DATA WITH HRTNG





MANY MORE ALGORITHMS AVAILABLE!

OFF-BY-ONE
2025

VALID PE HEADER IN SIGHT

```
db   50h ; P
db   45h ; E
db     0
db     0
db   4Ch ; L
db     1
db     0
db     0
db   48h ; H
db   80h
db  0DAh
db   5Ch ; \
db     0
db     0
db     0
```

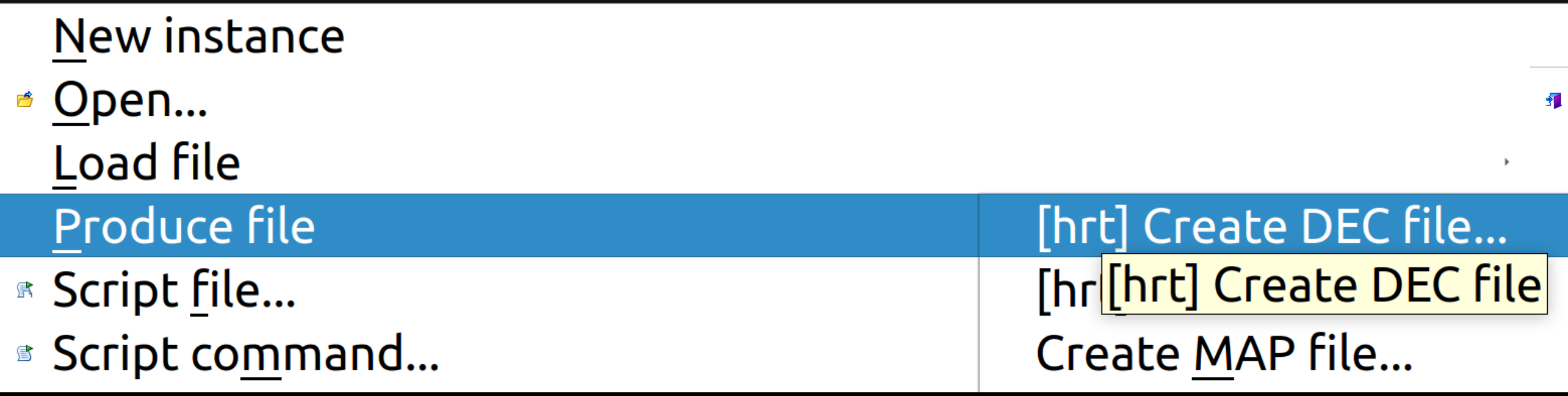


```
dd 4550h                ; Signature
dw 14Ch                 ; FileHeader.Machine
dw 0                    ; FileHeader.NumberOfSections
dd 5CDA8048h            ; FileHeader.TimeDateStamp
dd 0                    ; FileHeader.PointerToSymbolTable
dd 0                    ; FileHeader.NumberOfSymbols
dw 0E0h                 ; FileHeader.SizeOfOptionalHeader
dw 2102h                ; FileHeader.Characteristics
dw 10Bh                 ; OptionalHeader.Magic
db 8                    ; OptionalHeader.MajorLinkerVersion
db 0                    ; OptionalHeader.MinorLinkerVersion
dd 1BDC0h               ; OptionalHeader.SizeOfCode
dd 0F80h                ; OptionalHeader.SizeOfInitializedDat
dd 0                    ; OptionalHeader.SizeOfUninitializedD
dd 7FB8h                ; OptionalHeader.AddressOfEntryPoint
```



OFF-BY-ONE
2025

EXPORT DECRYPTED RESULTS



OFF-BY-ONE
2025

ERROR WHILE LOADING

Address	Ordinal	Name
004002A4	1997894001	ADVAPI32_1997894001
004002B0	985314727	ADVAPI32_985314727
004002B8	1828761019	ADVAPI32_1828761019
004002D0	389343560	GDI32_389343560
004002D4	266861538	GDI32_266861538
004002D8	1773711016	GDI32_1773711016
004002E4	1815244348	GDI32_1815244348
004002F8	337381605	GDI32_337381605
004002FC	1847517963	GDI32_1847517963

JUNK IMPORT NAMES



OFF-BY-ONE
2025

ANALYZING THE LOADER CODE

```
v31 = sub_A9D8(Flink_1, 0x5C2D1A97, 0);
Flink = sub_A9D8(Flink_1, 0xE0762FEB, 0);
v7 = sub_A9D8(Flink_1, 0xFA8303FB, 0);
v32 = sub_A9D8(Flink_1, -1513862064, 0);
sub_A9D8(Flink_1, 0x3F6F38C, 0);
v34 = 0;
if ( v32(-1, 26, &v34, 4, v24) )
```

LARGE NUMBER IN 2ND ARGUMENT
RETURN VALUE: FUNCTION POINTER



OFF-BY-ONE
2025

ANALYZING THE LOADER CODE

```
v4 = a1 + *(a1 + 0x3C);
result = *(v4 + 0x78);
```

OPERATIONS WITH PE HEADER

```
for ( i = -1; *v1; v2 = *v1 )
{
  i ^= v2;
  for ( j = 7; j >= 0; --j )
    i = -(i & 1) & 0xEDB88320 ^ (i >> 1);
  ++v1;
}
```

CRC32 HASH CALCULATION



OFF-BY-ONE
2025

API HASHING TECHNIQUE

Imagine you want to stealthily use
the NtProtectVirtualMemory function

NtProtectVirtualMemory → Calculate CRC32 hash → **0x5C2D1A97**



OFF-BY-ONE
2025

API HASHING TECHNIQUE

NtProtectVirtualMemory ⟶ **0x5C2D1A97**

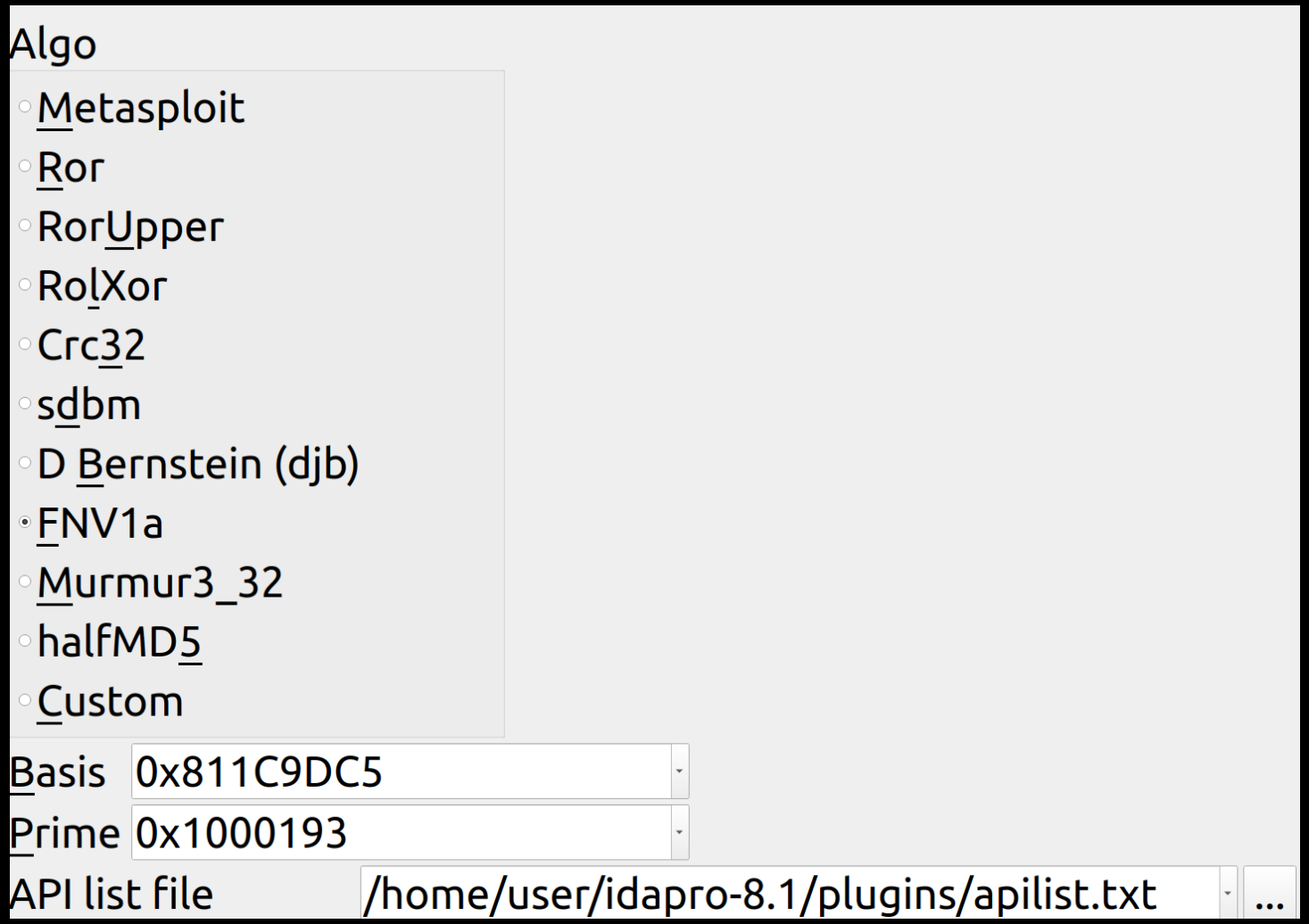
Iterate through API function names at runtime:

```
CRC32(A_SHAFinal) ==  0xbc661862 != 0x5C2D1A97
CRC32(A_SHAInit) ==  0x70417c04 != 0x5C2D1A97
 ...
CRC32(NtProtectVirtualMemory) == 0x5C2D1A97
```



OFF-BY-ONE
2025

HOW HRTNG CAN HELP



MOST COMMON HASHING ALGORITHMS IMPLEMENTED



OFF-BY-ONE
2025

HOW HRTNG CAN HELP

```
// NtProtectVirtualMemory
NtProtectVirtualMemory = sub_A9D8(Flink_1, 0x5C2D1A97, 0);
/NTSTATUS (__stdcall *NtProtectVirtualMemory)(HANDLE, PVOI
N    0: 0004 ^0.4        HANDLE ProcessHandle;
B    1: 0004 ^4.4        PVOID *BaseAddress;
/    2: 0004 ^8.4        SIZE_T *NumberOfBytesToProtect;
N    3: 0004 ^12.4       ULONG NewAccessProtection;
/    4: 0004 ^16.4       PULONG OldAccessProtection;
S  RET 0004 eax          NTSTATUS;
P  TOTAL STKARGS SIZE: 20
```

TYPES AND ARGUMENT NAMES RECOVERED AUTOMATICALLY



OFF-BY-ONE
2025

SEARCH FOR PE SIGNATURES

```
for ( i = sub_A8ED(); ; --i )
{
  if ( *i == 'ZM' )
  {
    n0x400 = *(i + 60);
    if ( n0x400 < 0x400 && *(n0x400 + i) == 'EP' )
      break;
  }
}
```

Conclusion: malware loads itself by using PE headers



OFF-BY-ONE
2025

WHAT TO DO WITH THE HEADER

1. Map the executable file at the correct address.

2. Find out what library functions are imported by the malware.



OFF-BY-ONE
2025

DEALING WITH IMPORTS

```
struct IMAGE_IMPORT_DESCRIPTOR {
    DWORD   OriginalFirstThunk; // stores function names
    DWORD   TimeDateStamp;
    DWORD   ForwarderChain;
    DWORD   Name; // library name
    DWORD   FirstThunk; // stores function addresses
}
```

Import structure tampering observed

```
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12768h, 470h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12832h, 4A0h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 128FEh, 43Ch>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12DDAh, 304h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12DFEh, 420h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12F08h, 2C0h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12F26h, 428h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 12F5Ah, 430h>
IMAGE_IMPORT_DESCRIPTOR <<0>, 0, 0, 13004h, 2A0h>
```



OFF-BY-ONE
2025

AUTOMATIC TYPE RECOGNITION

```
; NTSTATUS (__stdcall *NtQueryInformationProcess)(HANDLE ProcessHandl
NtQueryInformationProcess dd 0A5C44C50h
; NTSTATUS (__stdcall *NtQueryInformationThread)(HANDLE ThreadHandle,
NtQueryInformationThread dd 31133574h
; NTSTATUS (__stdcall *NtQueryObject)(HANDLE Handle, OBJECT_INFORMATI
NtQueryObject    dd 59B1E5ADh
; NTSTATUS (__stdcall *)(HANDLE Handle)
```

Custom loading procedure:

- Import descriptors refer to array of hashes
- Loader resolves each hash, overwriting it with a function pointer



OFF-BY-ONE
2025

AUTOMATIC TYPE RECOGNITION

```
0xA5C44C50 &NtQueryInformationProcess
0x31133574 &NtQueryInformationThread
0x59B1E5AD &NtQueryObject
0x0D09C750 &NtClose
0x62EA7CB7 &NtSetInformationFile
```



OFF-BY-ONE
2025

AUTOMATIC TYPE RECOGNITION

```
; NTSTATUS (__stdcall *NtQueryInformationProcess)(HANDLE ProcessHandl
NtQueryInformationProcess dd 0A5C44C50h
; NTSTATUS (__stdcall *NtQueryInformationThread)(HANDLE ThreadHandle,
NtQueryInformationThread dd 31133574h
; NTSTATUS (__stdcall *NtQueryObject)(HANDLE Handle, OBJECT_INFORMATI
NtQueryObject    dd 59B1E5ADh
; NTSTATUS (__stdcall *)(HANDLE Handle)
```

Thanks to hrtng, we have automatic function argument recovery!



OFF-BY-ONE
2025

ENTRY POINT FUNCTION

```
or      esi, [ecx+797h]
shr     esi, cl
xor     esi, eax
imul    esi, ecx
shrd    esi, edi, 0Ch
shld    esi, eax, 1Dh
sbb     esi, 0B853AC02h
adc     esi, 0CA6C6758h
xor     esi, [ecx+1144h]
imul    esi, esp
mov     esi, ebp
mov     esi, 94E0D0Eh
or      esi, 0A7F854DCh
add     esi, [ecx+0C86h]
```

"Fill nops" function



```
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
```



OFF-BY-ONE
2025

OPPOSITE JUMPS

```
loc_402E40:

                ja      loc_403161
                jbe     loc_403161
loc_403161:

                jz      loc_402EFC
                jnz     loc_402EFC
loc_402EFC:

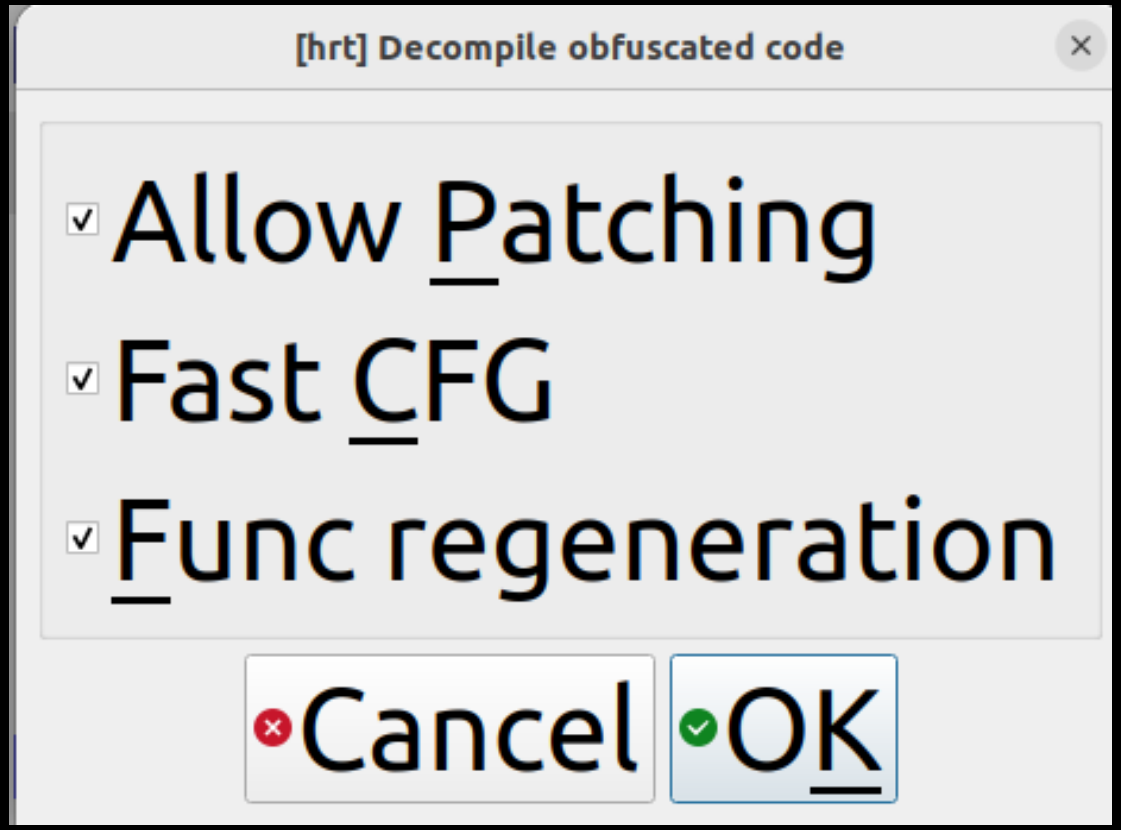
                push    eax
                jo      loc_402FEB
                jno     loc_402FEB
```

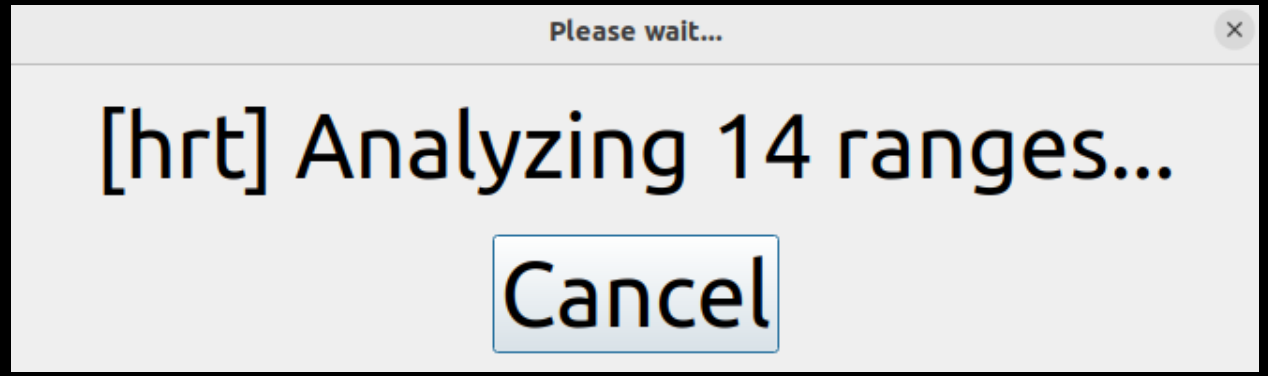
One of the two conditional jumps is always executed



OFF-BY-ONE
2025

HRTNG TO THE RESCUE AGAIN





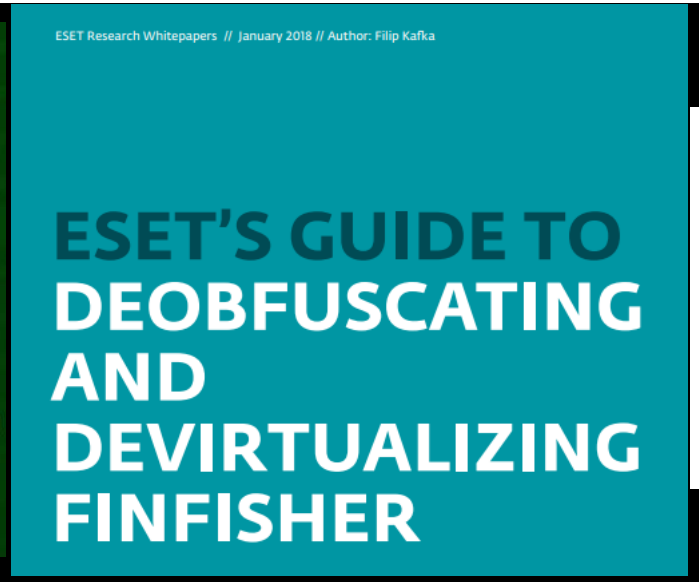


OFF-BY-ONE
2025

RESULT: FINSPY VM ENGINE

```
ctx->cmd_complete = n_4210865_;
ctx->cmd_setBlk = &unk_40412E;
ctx->cmd_jmp = &unk_40405F;
ctx->cmd_nextBlk = (&unk_4036ED + 2406);
ctx->psha1_stk = &n_4206384__10_8;
ctx->savedStack = &n_4206384__10_8;
qmemcpy(&ctx->blkDec, ctx->blk, sizeof(ctx->blkDec));
XOR_fromSecondD(&ctx->blkDec, 24, key);
sub_403766();
__asm { jmp     dword ptr [eax+ecx*4] }
return result;
```





A WALK-THROUGH TUTORIAL, WITH CODE, ON STATICALLY UNPACKING THE FINSPY VM: PART ONE, X86 DEOBFUSCATION

January 23, 2018 / Rolf Rolles

RECOGNIZING FUNCTIONS

```
# TODO for every sample:
# specify numerical opcode
# of each instruction
class VMOpcodes(Enum):
    ZeroVirtReg =
    Jump =
    MovVirtRegToX86Reg =
    ExecX86Code =
    Jno =
    ShlVirtReg =
    Je =
```

```
insnHandlers    dd offset sub_4041E6
                dd offset sub_404485
                dd offset sub_4044EC
                dd offset sub_40454C
                dd offset sub_4045EC
                dd offset sub_40465F
                dd offset sub_404778
                dd offset sub_4047F3
                dd offset sub_404891
                dd offset sub_4048F2
```

Opcode: index in this instruction handler table
Need to match handler functions with instructions
(e.g. sub_404485 does a SHL)

RECOGNIZING FUNCTIONS

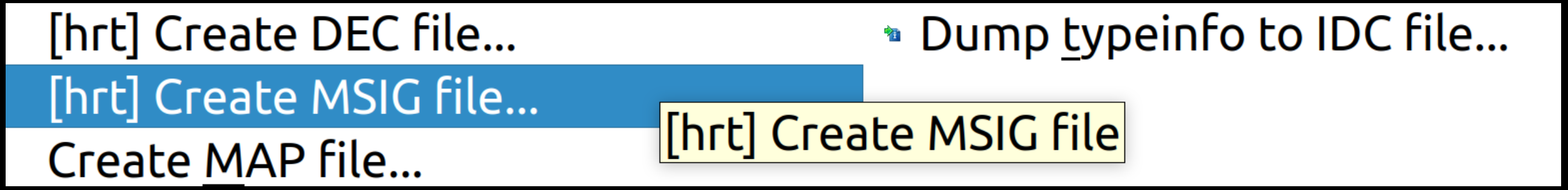
IDA's function recognition engine:

Based on assembly, may not work well with obfuscated code

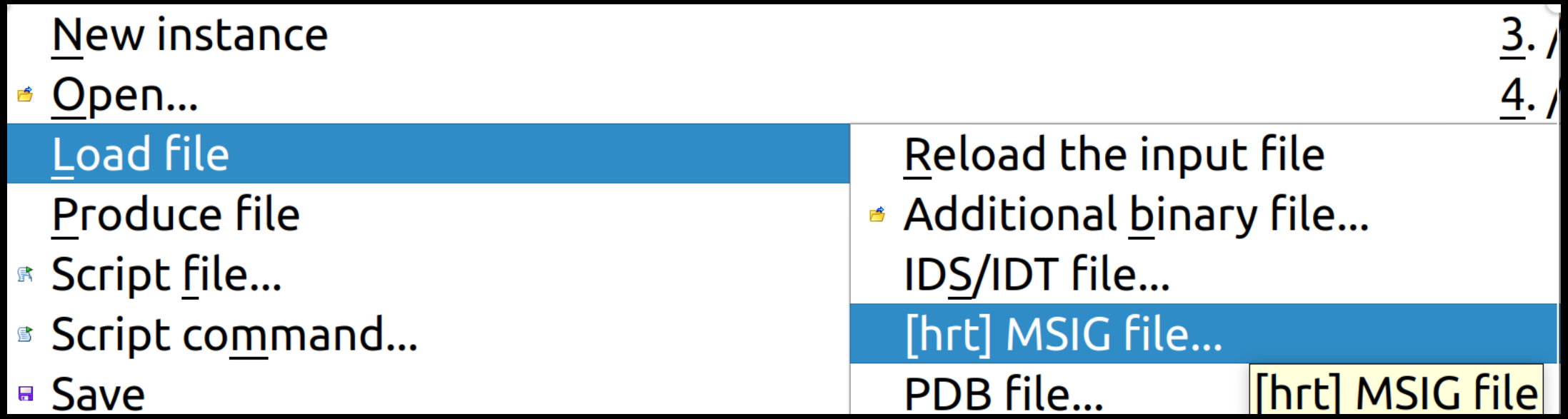
hrtng's function recognition engine:

based on microcode, works better with obfuscated code

RECOGNIZING FUNCTIONS



Step 1: export function names from analyzed sample



Step 2: load created signatures into another sample

AFTER APPLYING MSIGS

```
insnHandlers    dd offset sub_4041E6
                dd offset sub_404485
                dd offset sub_4044EC
                dd offset sub_40454C
                dd offset sub_4045EC
                dd offset sub_40465F
                dd offset sub_404778
                dd offset sub_4047F3
                dd offset sub_404891
                dd offset sub_4048F2
```

```
insnHandlers    dd offset cmd_DoJc
                dd offset cmd_Shl
                dd offset cmd_Jb
                dd offset cmd_Jle
                dd offset cmd_AddReg
                dd offset cmd_Exec
                dd offset cmd_Jna
                dd offset cmd_StReg
                dd offset cmd_Jns
                dd offset cmd_Jne
                dd offset cmd_GetReg
```

```
class VMOpcodes(Enum):
    ZeroVirtReg = 21
    Jump = 0
    MovVirtRegToX86Reg = 29
    ExecX86Code = 5
    Jno = 24
    ShlVirtReg = 1
    Je = 11
    Jns = 8
```

FINAL OUTCOME

Espionage implant capabilities:

- Filesystem control
- Registry key management
- Keystroke interception
- PGP key theft

- And even more things!

WHAT WE'VE DONE TODAY

Code decryption

API Hashing analysis

Control flow deobfuscation

Function recognition

Virtual machine deobfuscation

All these things would have been done much slower without hrtng!



Download hrtng
using this QR code!
github.com/KasperskyLab/hrtng

THANK YOU

Many thanks to
Sergey Belov,
the creator and main
contributor of hrtng!